

Container-Virtualisation mit Docker, Schlüsselkomponenten

Kernbotschaft:

Docker ist nicht nur ein Tool, um Container zu starten, sondern ein modulares Ökosystem aus spezialisierten Werkzeugen für die Entwicklung, Verteilung und Orchestrierung. Der Hintergrundgrunddienst (Daemon), hat eine Kernaufgabe: Verwaltung der privaten Schlüssel im Arbeitsspeicher. Normalerweise sind die privaten Schlüssel auf der Festplatte, was bei hohem Remote-Verkehr zu Engpässen führen kann.

Stichpunkte:

Weg von der monolithischen "Alles-auf-einem-Server"-Struktur.

Hin zu verteilten, containerisierten Microservices.

Herausforderung: Je mehr Container, desto komplexer das Management. Docker-Komponenten teilen sich diese Aufgaben auf.

Container-Virtualisation mit Docker, Schlüsselkomponenten

Was macht die Container-Engine

Stichpunkte:

Aufgabe: Erstellen, Ausführen und Verwalten einzelner Container auf einem Host (z. B. Rocky Linux).

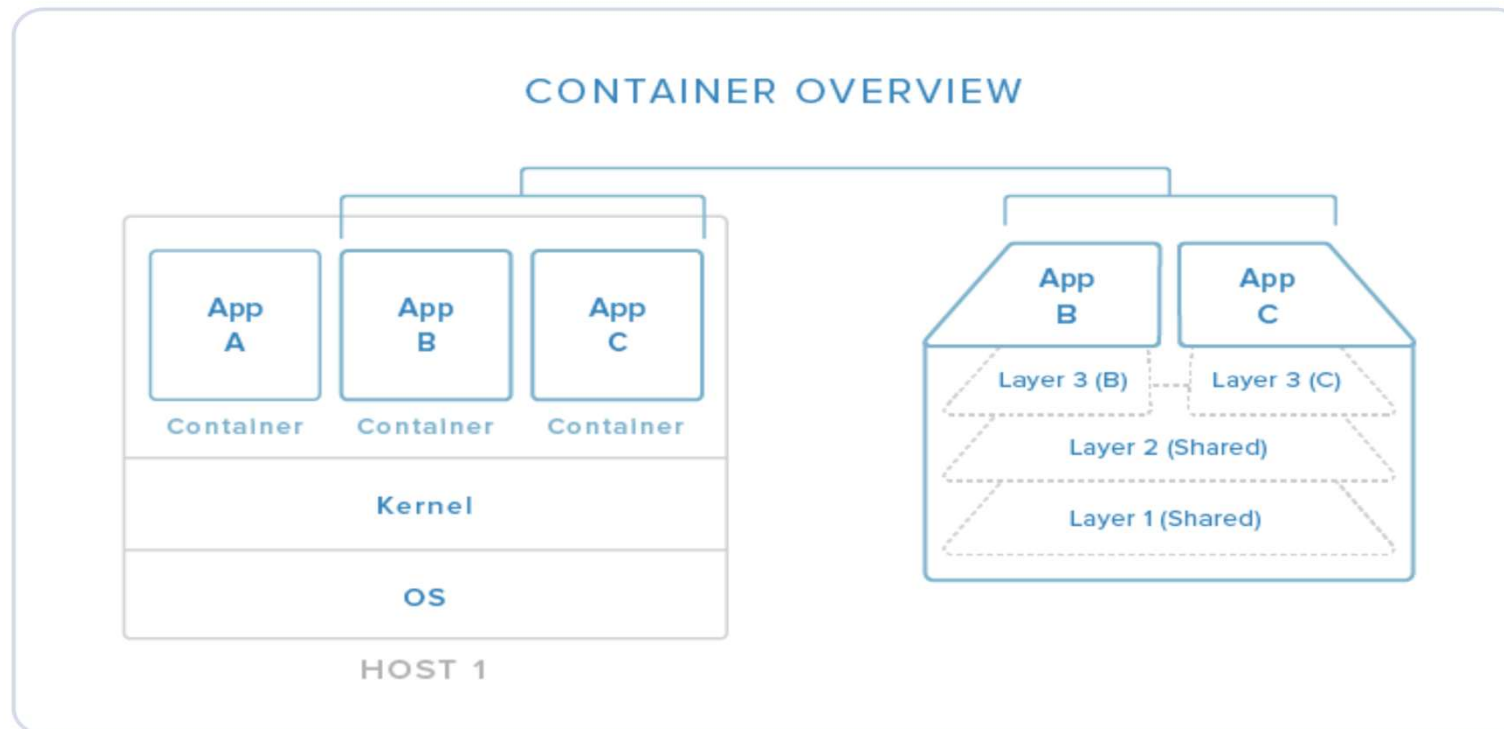
Vorteil: Isolierte Laufzeitumgebungen, schnelle Startzeiten, geringer Ressourcenverbrauch im Vergleich zu klassischen VMs.

Einschränkung: Docker allein verwaltet Container nur auf einem physischen oder virtuellen Rechner.

Container-Virtualisation mit Docker, Schlüsselkomponenten

Was macht die Container-Engine?

Quelle: <https://www.digitalocean.com/community/tutorials/the-docker-ecosystem-an-introduction-to-common-components>



Container-Virtualisation mit Docker, Schlüsselkomponenten

Wie finden sich die Container? (Service Discovery)

Stichpunkte:

Das Problem: In einer dynamischen Cloud-Umgebung ändern Container ständig ihre IP-Adressen (durch Skalierung oder Neustarts).

Die Lösung: Ein zentrales "Telefonbuch" für das System.

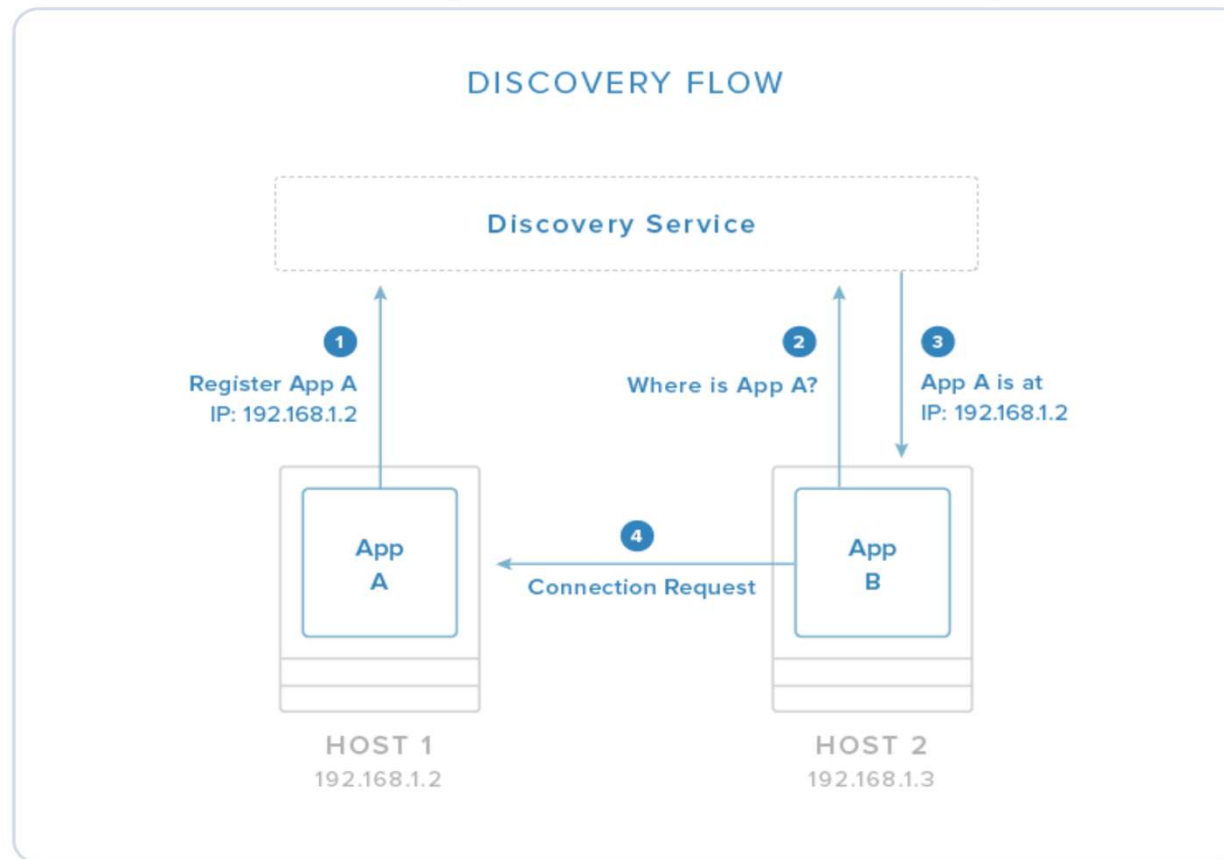
Funktionsweise: Neue Container registrieren sich automatisch beim Start. Andere Container fragen das Telefonbuch ab, um zu wissen, wo der gewünschte Dienst erreichbar ist.

Bekannte Tools: Consul, etcd, ZooKeeper.

Container-Virtualisation mit Docker, Schlüsselkomponenten

Wie finden sich die Container? (Service Discovery)

Quelle: <https://www.digitalocean.com/community/tutorials/the-docker-ecosystem-an-introduction-to-common-components>



Container-Virtualisation mit Docker, Schlüsselkomponenten

Brücken bauen: Container-Netzwerke

Stichpunkte:

Aufgabe: Ermöglicht die Kommunikation zwischen Containern, die über verschiedene physische Server hinweg verteilt sind.

Overlay-Netzwerke: Erstellen ein virtuelles, flaches Netzwerk über alle beteiligten Server hinweg. Für die Container sieht es so aus, als würden sie im selben lokalen Switch stecken.

Sicherheit: Isolation von Netzwerken (z. B. darf der Datenbank-Container nur mit dem Backend reden, aber nicht direkt aus dem Internet erreichbar sein).

Container-Virtualisation mit Docker, Schlüsselkomponenten

Der Dirigent des Orchesters: Scheduler & Orchestratoren

Stichpunkte:

Aufgabe: Automatisches Ressourcenmanagement in einem Server-Cluster.

Wie funktioniert es? Der Admin sagt: "Ich brauche 3 Instanzen von Web-App X". Der Scheduler prüft die Auslastung der Server (RAM/CPU) und entscheidet eigenständig, auf welchen Servern die Container gestartet werden.

Features: Self-Healing (automatische Neustarts bei Absturz) und Skalierung.

Bekannte Tools: Kubernetes (K8s), Docker Swarm, Nomad.

Container-Virtualisation mit Docker, Zusammenfassung

Das Zusammenspiel im Ökosystem

Stichpunkte:

Docker CE: Baut und läuft den Container.

Networking Tools: Verbinden die Container über Servergrenzen hinweg.

Service Discovery: Hilft den Containern, sich gegenseitig zu finden.

Scheduler (Kubernetes): Verwaltet, wo die Container laufen und hält sie am Leben.

Container-Virtualisation mit Docker, Zusammenfassung

Das Zusammenspiel im Ökosystem

Stichpunkte:

Docker CE: Baut und läuft den Container.

Networking Tools: Verbinden die Container über Servergrenzen hinweg.

Service Discovery: Hilft den Containern, sich gegenseitig zu finden.

Scheduler (Kubernetes): Verwaltet, wo die Container laufen und hält sie am Leben.

Container-Virtualisation mit Docker, Containerisierung

Was ist Containerisierung überhaupt?

Kernbotschaft: Containerisierung ist eine Methode der Virtualisierung auf Betriebssystem-Ebene (OS-level virtualization), nicht auf Hardware-Ebene.

Stichpunkte:

Im Gegensatz zu VMs wird keine Hardware (CPU, RAM, Festplatte) virtuell nachgebaut.

Alle Container teilen sich den gleichen Linux-Kernel des Host-Systems.

Ein Container ist aus Sicht des Host-Systems im Grunde nur ein isolierter, ganz normaler Prozess.

Container-Virtualisation mit Docker, Containerisierung

Wie isoliert Linux die Container?

Kernbotschaft: Docker hat Container nicht erfunden, sondern nutzt tief im Linux-Kernel verankerte Standard-Features aus, um die Isolation zu erzwingen.

Die 3 Kern-Features:

Namespaces (Namensräume): Bestimmen, was ein Container sehen darf.

Control Groups / cgroups: Bestimmen, wie viel ein Container nutzen darf (Ressourcenlimitierung).

Chroot / Pivot_root: Bestimmt, wo der Container auf der Festplatte eingesperrt ist.

Container-Virtualisation mit Docker, Namespaces

Stichpunkte:

Ein Namespace sorgt dafür, dass ein Prozess denkt, er wäre allein auf dem System.

Wichtige Namespace-Typen, die Docker anlegt:

pid (Process ID): Der Container sieht nur seine eigenen Prozesse. Seine Hauptanwendung hat im Container die PID 1 (obwohl sie auf dem Host-System z. B. die PID 4829 hat).

net (Network): Jeder Container bekommt sein eigenes virtuelles Netzwerk-Interface und eigene IP-Adressen.

mnt (Mount): Der Container hat sein eigenes isoliertes Dateisystem.

uts (Hostname): Der Container kann einen eigenen Computernamen haben.

user: Ermöglicht es, dass ein Benutzer innerhalb des Containers root (Admin) ist, auf dem echten Host System aber nur ein unprivilegierter Standard-Nutzer bleibt.

Container-Virtualisation mit Docker, cgroups & Chroot

Titel: cgroups & Chroot: Ressourcen und Gefängnis

Stichpunkte:

Control Groups (cgroups):

Ohne cgroups könnte ein einzelner Amok laufender Container den gesamten RAM des Rocky Linux Servers auffressen und das System lahmlegen.

cgroups limitieren hart die Hardware: "Dieser Container darf maximal 512 MB RAM und 10% CPU nutzen."

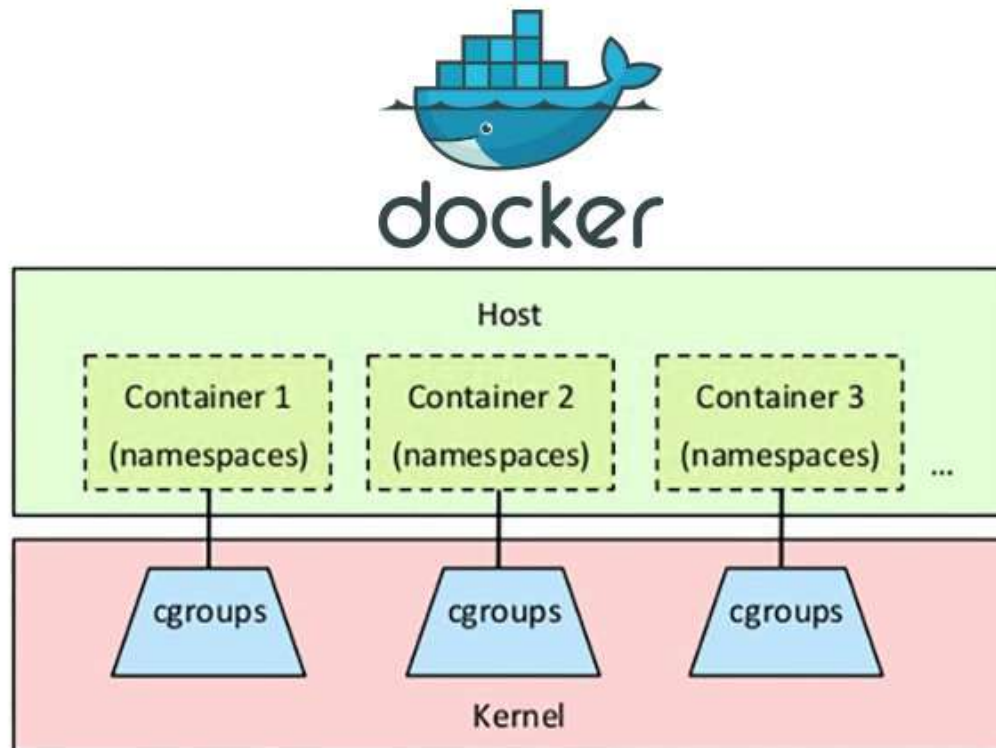
chroot / pivot_root:

Verschiebt das scheinbare Wurzelverzeichnis (/) für den Container. Der Container kann physisch nicht aus seinem Ordner ausbrechen, um Systemdateien des echten Servers zu sehen oder zu manipulieren.

Container-Virtualisation mit Docker, Containerisierung

Wie isoliert Linux die Container?

Quelle: <https://www.containex.com>, <https://dyma.fr/blog/docker-et-ses-conteneurs/>



Container-Virtualisation mit Docker, Union File Systems (UFS)

Docker Images und das Schichten-Dateisystem (UFS)

Stichpunkte:

Docker-Images bestehen aus mehreren übereinandergestapelten, schreibgeschützten Schichten (Read-Only Layers).

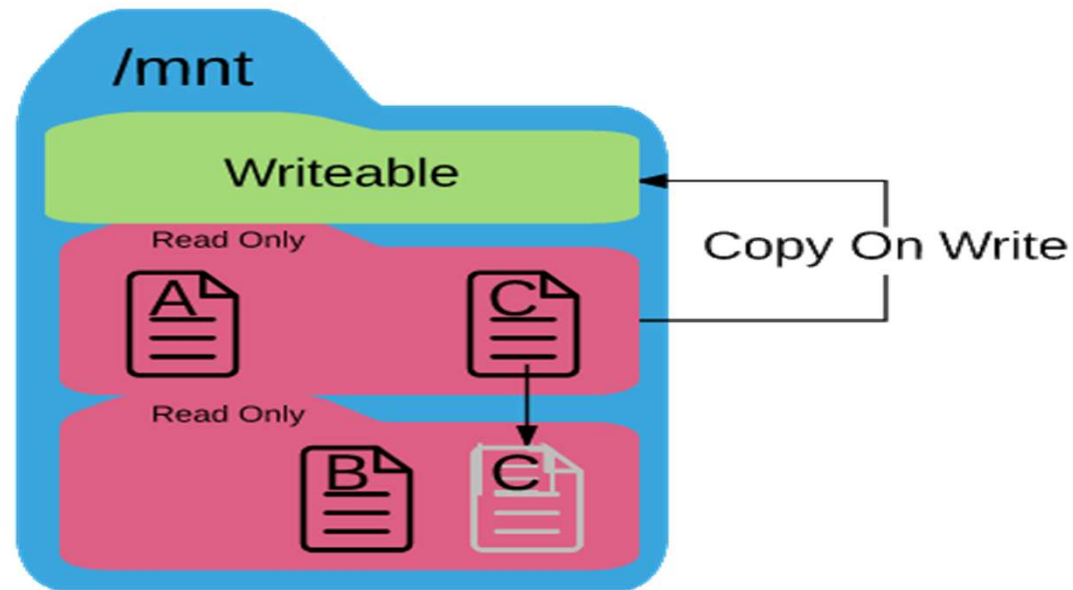
Beispiel: Schicht 1 (Basis-Betriebssystem Rocky Linux) → Schicht 2 (Node.js Installation) → Schicht 3 (Angular-Code).

Der Clou: Startet man den Container, legt Docker eine hauchdünne, beschreibbare Schicht ganz oben drauf (Read-Write Container Layer).

Alle Änderungen, die der Container im Betrieb macht, landen nur in dieser obersten Schicht. Das spart enorm viel Speicherplatz, da sich 10 Container dieselben Basis-Schichten teilen können.

Container-Virtualisation mit Docker, Union File Systems (UFS)

Docker Images und das Schichten-Dateisystem (UFS)



Container-Virtualisation mit Docker, Container vs. VMs (Fazit)

Container vs. VMs im direkten Vergleich

Vorteile von Containern:

Extrem leichtgewichtig: Starten in Millisekunden statt Minuten.

Hohe Dichte: Man kann hunderte Container auf einem Server laufen lassen, wo sonst nur wenige VMs Platz hätten.

Portabilität: Läuft es in Docker auf dem Entwickler-Laptop, läuft es exakt genau so auf dem Rocky-Linux-Server in der Cloud.

Nachteile / Risiken:

Geringere Isolation als VMs: Da der Kernel geteilt wird, kann eine Sicherheitslücke im Linux-Kernel potenziell alle Container auf diesem Server gefährden.